



# pandas 学习

## Elegant $\text{\LaTeX}$ 经典之作

作者: Ethan Deng & Liam Huang

组织: Elegant $\text{\LaTeX}$  Program

时间: April 9, 2022

版本: 4.3

自定义: 信息



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡

# 目录

|            |
|------------|
| 第 1 章 基础知识 |
|------------|

# 第 1 章 基础知识



**笔记** 在 Jupyter Notebook 中，`?` 是内置的自省符号，用于快速查看对象的信息。

```
# 查看对象的类型、文档、属性
x = [1, 2, 3]
x?
# 查看函数的参数和文档
import numpy as np
np.array?
# 使用 ?? 查看源代码
np.array??
```

`?` 显示对象的基本信息和文档字符串；`??` 在此基础上进一步显示源代码。常用于快速了解不熟悉的函数用法，是 Jupyter 中最常用的调试和学习工具之一。

## 东邪的 tip 1.1

`is` 只用于判断是不是 `null`

## 定义 1.1 (可变对象与不可变对象)

在 Python 中，对象分为可变 (mutable) 和不可变 (immutable) 两类。

不可变对象：创建后其值不能被修改，修改时会创建一个新对象。常见类型有：`int`、`float`、`str`、`tuple`、`bool`。

可变对象：创建后其值可以直接修改，不会创建新对象。常见类型有：`list`、`dict`、`set`。

例如：

```
# 不可变: str
s = "hello"
s[0] = "H" # 报错，字符串不可修改

# 可变: list
lst = [1, 2, 3]
lst[0] = 99 # 合法，列表可直接修改
```



## 定义 1.2 (import 导入模块)

在 Python 中，使用 `import` 语句导入模块或包，常见用法有三种：

```
# 导入整个模块
import numpy

# 导入模块并起别名（最常用）
import numpy as np

# 从模块中导入特定函数或类
from numpy import array

# 从模块中导入所有内容（不推荐，容易命名冲突）
```

```
from numpy import *
```

导入后即可使用模块中的函数和类，例如 `np.array()`、`np.mean()` 等。



### 定义 1.3 (鸭子类型 (Duck Typing))

鸭子类型是 Python 的一种编程思想：不关心对象是什么类型，只关心它是否具有所需的方法或属性。名称来自“如果它走起来像鸭子，叫起来像鸭子，那它就是鸭子”。

```
class Duck:
    def quack(self):
        print("嘎嘎嘎")

class Person:
    def quack(self):
        print("我在模仿鸭子")

def make_it_quack(obj):
    obj.quack() # 不管传入什么类型，只要有 quack 方法就行

make_it_quack(Duck()) # 嘎嘎嘎
make_it_quack(Person()) # 我在模仿鸭子
```

与 Java 等强类型语言不同，Python 无需声明类型或继承同一父类，只要对象具备所需行为即可直接使用。



### 定义 1.4 (参数传递)

Python 中函数的参数传递遵循传对象引用的规则：传递的既不是值的拷贝，也不是变量本身，而是对象的引用。其行为取决于对象是否可变：

不可变对象（如 `int`、`str`）：函数内部修改不影响外部变量，相当于传值。

可变对象（如 `list`、`dict`）：函数内部修改会影响外部变量，相当于传引用。

```
# 不可变对象：外部不受影响
def f(x):
    x = 10

a = 5
f(a)
print(a) # 仍然是 5

# 可变对象：外部受影响
def g(lst):
    lst.append(99)

b = [1, 2, 3]
g(b)
print(b) # [1, 2, 3, 99]
```

